Sei un ingegnere senior esperto di NVIDIA CUDA-Q (v0.9+), quantum computing applicato alla logistica industriale, Python 3.11+, crittografia post-quantum (NIST FIPS 203/204/205) e sistemi MES/WMS/YMS/TMS/ECS/SDM. Ti allego un file consolidato chiamato "JOKER 80" che contiene 21 "calcoli quantistici" per lo stabilimento Barilla di Novara. Il file mescola tre livelli:
(a) codice Python/CUDA-Q che dovrebbe essere eseguibile,
(b) report JSON di output scritti a mano (probabilmente non rigenerati),
(c) commenti descrittivi e metadati.

Rispondi SEMPRE in italiano, in modo tecnico, con esempi di codice corretti e con tabelle quando utile. Non inventare: se un'informazione manca nel file, scrivi "NON DEDUCIBILE DAL FILE". Se ricalcoli un valore, mostra il calcolo.

Il lavoro è organizzato in QUATTRO FASI. Esegui tutte e quattro in ordine, senza saltarne nessuna.

==========
FASE 1 — RICOSTRUZIONE DELLA LOGICA (cosa il file VUOLE fare)
==========
Per OGNUNO dei 21 calcoli, compila una riga di questa tabella:
| ID | Nome calcolo | Obiettivo industriale (1 frase) | Input di campo (variabili PLC/telemetria) | Output atteso (metrica principale + metriche dettagliate) | Formula matematica o quantistica che SAREBBE stata usata (deducila dal codice e dai commenti) | Reparto/Modulo (MIP/WMS/TMS/YMS/ECS/SDM) |

Poi, in fondo alla tabella, scrivi un paragrafo di sintesi:
- Qual è la "catena logica" end-to-end che collega i 21 calcoli (Inbound → Magazzino → Outbound → IoT/Macchine)?
- Quali calcoli sono ridondanti o sovrapposti tra loro?
- Quali calcoli non hanno una vera giustificazione quantistica (cioè potrebbero essere fatti in modo classico senza perdita)?

==========
FASE 2 — AUDIT COMPLETO DEGLI ERRORI (check esaustivo)
==========
Esegui un audit SISTEMATICO. Non fermarti ai primi errori: cerca in TUTTE le categorie elencate sotto. Per ogni errore trovato, riporta:
- ID calcolo e numero di riga (o nome funzione)
- Categoria (una delle 10 sotto)
- Descrizione sintetica
- Severità: BLOCCANTE / GRAVE / MEDIO / LIEVE
- Correzione proposta (codice o formula)

Categorie da verificare (obbligatorie, nessuna esclusa):

C1. API CUDA-Q
- cudaq.set_target(...) valido? ("nvidia", "custatevec", "tensornet", "ionq", "dwave", ecc.)
- cudaq.x.ctrl / cudaq.rz.ctrl / cudaq.ctrl(...) : sintassi corretta per la versione dichiarata (0.9+)?

- cudaq.swap(...) esiste come funzione top-level?
- Uso di cudaq.mz(...) DENTRO il kernel + cudaq.sample(...) FUORI: pattern coerente? Restituisce i bit o li misura due volte?
- Passaggio di list[float] come argomento di @cudaq.kernel: supportato? (probabile crash JIT)
- Misura di registri con lunghezza diversa da quella attesa (es. qvector(4) ma poi si indicizza q[4] o si legge stringa di 3 bit)
- Uso di np.* dentro il kernel (vietato: solo costanti o parametri)
- Gate applicati a qubit non entangled ma dichiarati "OBBLIGATORIO"

C2. Sintassi Python / tipi
- Type hint non validi (list[float] nei kernel, List importato e mai usato, Dict/Any non usati)
- Cast impliciti pericolosi (stringa SSCC → int → perdita zero iniziale)
- Divisioni per zero potenziali (baie_totali=0, volume=0, ecc.)
- Overflow / perdita precisione su interi > 2^53 in JSON
- Variabili non definite, import mancanti, funzioni duplicate
- Uso di `+` unario inutile, confronti ridondanti, range pericolosi

C3. Coerenza input ↔ output (JSON nel report vs codice)
- Il JSON di output nel commento corrisponde DAVVERO ai valori che la funzione Python produce con gli input dichiarati? Ricalcola a mano e confronta. Elenca TUTTI i disallineamenti.
- Le "metriche dettagliate" sono derivate dalle formule del codice o sono inventate?
- I timestamp sono coerenti?
- I codici di stato/allarme (es. 200, 414, 88, 99) hanno una logica o sono arbitrari?

C4. Correttezza matematica / fisica
- Le formule di congestione, VaR, efficienza, saturazione volumetrica, fattore di sicurezza, energia ground state, ecc. sono dimensionalmente corrette?
- Le soglie (es. 0.85, 70%, 18 MPa, 45°C, 1.5) sono giustificate o hard-coded senza motivo?
- Le unità di misura sono coerenti (Hz vs RPM, bar vs MPa, µm vs mm)?
- I valori di output sono fisicamente plausibili? (es. 150 Hz da 48 RPM, 9.2% di saturazione con 33 colli in 86 m³, 27 plt/h di scarti su 280)
- Le conversioni (rpm→Hz, N→MPa, g→m/s²) sono fatte bene?

C5. Correttezza quantistica (sostanza, non solo sintassi)
- Il circuito implementa DAVVERO l'algoritmo dichiarato nel nome? (QBM, QMC, QAOA, Quantum Walk, QGNN, QFT, QSVM, VQE, QGT, Quantum K-Means, Quantum Knapsack, QEM, ZKP/ML-DSA)
- Ci sono entanglement reali o solo H + rotazioni + misura?
- I parametri variazionali sono ottimizzati o fissi?
- Il numero di qubit è sufficiente per l'algoritmo?
- Il "Quantum Amplitude Estimation" del Calcolo #2 è davvero QAE o è solo un H-Z-H su un qubit?
- Il "Quantum Fourier Transform" del Calcolo #14 è una QFT corretta

(rotazioni controllate + swap)?
- Il "Blind Signature" del Calcolo #11 è davvero una firma cieca o solo una fase controllata?
- La "ZKP / ML-DSA" del Calcolo #21 è implementata o solo nominata?
- Le "Post-Quantum Cryptography" (#5, #11, #21) usano davvero primitive PQC o solo SHA-3 + stringhe?
- Il Calcolo #9 (QGNN) con "pesi statici" è una GNN o un circuito fisso?
- Il Calcolo #3 (QUBO) è davvero un QAOA con ottimizzazione variazionale o un circuito a parametri fissi?

C6. Sicurezza / compliance dichiarata
- "NIS2 Art. 21", "FIPS 203/204", "ISO 19944", "Dilithium5/Falcon-1024" sono usati correttamente o solo come etichette?
- Il salt quantistico del Calcolo #5 ha davvero 256 bit di entropia o solo 4 bit (stringa "1111")?
- La "firma cieca" garantisce davvero anonimato/blindness?
- I dati personali (autisti, vettori) sono trattati secondo GDPR?
- Ci sono credenziali/hard-coded secrets?

C7. Robustezza / gestione errori
- Cosa succede se un input è 0, negativo, NaN, stringa vuota, fuori range?
- I try/except sono assenti? Le eccezioni CUDA-Q sono gestite?
- I fallback classici esistono se la QPU non è disponibile?
- I timeout / retry sono previsti?
- La simulazione è deterministica o dipende dal seed?

C8. Performance / scalabilità
- 21 chiamate a cudaq.set_target("nvidia") sono necessarie?
- 1000 shot per calcolo sono giustificati?
- I kernel vengono ricompilati a ogni chiamata o cachati?
- La complessità del circuito è compatibile con NIS2 / tempi reali?
- Ci sono colli di bottiglia (es. sample sincrono su 21 calcoli)?

C9. Manutenibilità / stile
- Duplicazione di codice (import, set_target, pattern identici)
- Nomi di funzioni/variabili coerenti?
- Commenti allineati al codice (o fuorvianti)?
- Magic numbers non documentati
- Assenza di test unitari
- Assenza di logging strutturato
- Assenza di type checking / linting

C10. Allineamento al dominio industriale
- I nomi di reparto/modulo (MIP, WMS, TMS, YMS, ECS, SDM) sono usati coerentemente?
- Le variabili PLC hanno nomi/unità plausibili?
- I flussi Inbound → Magazzino → Outbound → IoT sono rispettati?
- Ci sono incoerenze tra il testo descrittivo e il codice?

- I riferimenti a macchine reali (Bema Silkworm, Woodpecker, Raptor, SmartStore, LGV) sono coerenti con le funzioni?

Alla fine della FASE 2, produci una TABELLA RIASSUNTIVA:
| Categoria | N. errori | N. bloccanti | N. gravi | N. medi | N. lievi |
e un ELENCO NUMERATO di TUTTI gli errori trovati (non solo i primi), ordinato per severità decrescente.

==========
FASE 3 — RISOLUZIONE (fix concreti)
==========
Per ogni errore BLOCCANTE o GRAVE della FASE 2, fornisci:
- Il codice corretto (snippet pronto da incollare)
- La motivazione tecnica della correzione
- L'impatto sui risultati (cambia il valore di output? di quanto?)

Poi fornisci una VERSIONE CORRETTA E MINIMA di 3 kernel rappresentativi:
1. Un kernel che usa correttamente cudaq.ctrl(...) e cudaq.mz(...) senza doppia misura (es. Calcolo #1 QBM).
2. Un kernel che implementa una VERA QFT con swap corretti (Calcolo #14).
3. Un kernel che implementa un VERO QAOA con parametri variazionali ottimizzati (Calcolo #3 o #6), mostrando il loop di ottimizzazione con cudaq.observe o cudaq.vqe.

Infine, fornisci un TEMPLATE di "kernel corretto" generico che rispetti le best practice CUDA-Q 0.9+ (target, ctrl, mz, sample, parametri, niente list[float], niente np.* dentro il kernel).

==========
FASE 4 — RACCOMANDAZIONI STRATEGICHE
==========
Rispondi a queste domande:
1. Quali dei 21 calcoli hanno un VERO vantaggio quantistico e quali sono "quantum washing" (cioè solo rinominati ma classici)?
2. Quali calcoli andrebbero RISCRITTI da zero con un algoritmo quantistico reale (es. QAE vero, QAOA variazionale vero, VQE vero)?
3. Quali calcoli andrebbero invece RISCRITTI in classico (più veloce, più robusto, più mantenibile)?
4. Come strutturare il file in moduli separati (es. una classe per reparto, un runner comune, un config YAML per i parametri)?
5. Quali test unitari minimi andrebbero scritti per garantire la correttezza (es. test che il JSON di output corrisponda ai valori calcolati)?
6. Come garantire la compliance NIS2 reale (non solo dichiarata)?

==========
FORMATO DI RISPOSTA
==========
- Usa tabelle markdown per FASE 1 e FASE 2.

- Usa blocchi di codice ```python per FASE 3.
- Usa elenchi puntati per FASE 4.
- Non superare le 6000 parole totali, ma non tralasciare nessuna categoria di errore.
- Se un'informazione non è deducibile dal file, scrivilo esplicitamente.
- Non inventare numeri: se ricalcoli, mostra il calcolo.
- Alla fine della risposta, aggiungi una sezione "CHECKLIST FINALE" con 20 domande sì/no che riassumono i punti chiave dell'audit, così posso verificare rapidamente che tu abbia coperto tutto.